

CHIP 2026 Hackathon — Tim J5 (ITB)

Laporan Fase 2

Arsitektur Memori Ter-bank Bebas-Tabrakan (Konfigurasi C1)

Status: **PARTIAL** — bebas-tabrakan terbukti, timing Quartus masih belum terpenuhi, M10K belum tercapai
Branch: phase2-memory-banking · HEAD: 6aa515f · belum di-push ke main

Ditulis untuk: Tim J5 — laporan Bahasa Indonesia Fase 2 (belum ada versi Bahasa Inggris terpisah). Sumber kebenaran tetap file-file di repo (docs/results/result_phase2.md, docs/evidence/phase02-memory/); dokumen ini adalah rangkuman bacaan.

1. Tujuan dan Lingkup

Fase 2 membangun memori dan generator alamat yang bisa melayani sampai **8 lane paralel tanpa tabrakan bank**, sebagai persiapan untuk eksplorasi multi-lane di Fase 3. Sengaja datapath yang benar-benar disintesis dan diukur tetap **1 lane (L=1)** — konfigurasi **C1** — supaya hanya perubahan arsitektur memori yang terlihat pada angka Quartus, bukan tercampur dengan perubahan aritmetika atau pipa.

Catatan proses: Fase 2 dimulai sebelum kotak Approval Fase 1 dicentang, atas instruksi langsung tim. Dicatat eksplisit di sini dan di docs/results/result_phase2.md, bukan disembunyikan.

2. Masalah: Pem-bank-an In-place NTT

Algoritma NTT/INTT ML-KEM-768 bersifat *in-place*: alamat 0–255 adalah lokasi fisik tetap sepanjang seluruh 7 lapis, bukan diinterpretasi ulang per lapis. Pasangan butterfly ($j, j+len$) selalu berbeda tepat **1 bit**, tapi posisi bit pembeda itu **berubah per lapis** (dari bit-1 sampai bit-7 sepanjang 7 lapis).

Skema bank sederhana berbasis irisan bit tetap (mis. $bank = addr[k:0]$) **terbukti gagal** untuk sebagian besar lapis — dibuktikan lewat eksplorasi eksaustif, bukan diasumsikan gagal. Sebabnya: jendela bit tetap yang kecil tidak bisa mencakup posisi bit pembeda yang bergeser 1..7.

3. Skema yang Dipakai: XOR-group ("Skewed Storage")

Bit alamat 1–7 (bit-0 tidak pernah jadi pembeda pasangan) dibagi *round-robin* ke $\log_2(L)$ kelompok. Setiap bit keluaran nomor bank adalah **paritas-XOR** dari satu kelompok bit alamat. Karena setiap bit 1–7 masuk ke tepat satu kelompok, membalik bit manapun di antaranya **pasti** mengubah paritas kelompok itu — sehingga bank pasti berbeda, untuk lapis manapun.

Alamat fisik dalam bank ditentukan lewat pemetaan *offset-by-rank*: bit-bit alamat yang tersisa (di luar bit yang dipakai menentukan bank) diurutkan ulang menjadi offset dalam bank, dibuktikan bijektif terhadap alamat asal.

4. Bukti Bebas-Tabrakan (sebelum RTL ditulis)

Mengikuti disiplin "verifikasi sebelum optimisasi": rencana pengujian ditulis lebih dulu (docs/evidence/phase02-memory/test_plan.md), baru skema divalidasi komputasional, baru RTL dibangkitkan.

Yang dibuktikan	Metode	Hasil
Pasangan-sendiri: $bank(j) \neq bank(j+len)$, semua $L \in \{1,2,4,8\}$, semua 7 lapis, semua alamat	Python eksaustif (scripts/gen_bank_map.py)	0 tabrakan
Properti sama, pada isi ROM RTL sebenarnya ($L=8$, dibangkitkan)	Formal — SymbiYosys, BMC depth 1	LULUS
Bank seimbang: tiap bank persis berisi 256/L alamat	Python eksaustif	Seimbang, semua L
Pemetaan ($bank, offset$) $\leftrightarrow$ alamat bijektif	Python eksaustif	Bijektif, semua L
Kapasitas multi-lane: ≤ 2 akses/bank/siklus (kapasitas true-dual-port M10K)	Python eksaustif, dua skema pengelompokan lane	Maks. 2, semua L — hanya untuk pengelompokan 'contiguous-block'

Dua skema pengelompokan lane dicoba: 'contiguous-block' (lane ke-i memegang blok alamat berurutan) dan 'interleaved' (lane ke-i memegang alamat berselang-seling). Hanya 'contiguous-block' memenuhi batas 2 akses/bank/siklus untuk semua L; 'interleaved' terbukti GAGAL untuk $L=4$ dan $L=8$ (sampai 4 akses/bank pada siklus yang sama) — hasil ini **disimpan sebagai kontrol negatif terdokumentasi** di bank_scheme_exploration_2026-09-29.txt, bukan dihapus, supaya keputusan desain punya jejak bukti.

5. RTL yang Dihasilkan

Berkas	Isi
rtl/mem/bank_map_rom.sv	Dibangkitkan (scripts/gen_bank_map.py): $addr \rightarrow (bank, offset)$, 4 varian $L=1/2/4/8$ lewat generate-if elaborasi

Berkas	Isi
rtl/mem/poly_mem_banked.sv	Memori 256×12-bit ter-bank generik #(NUM_BANKS); tiap bank array terpisah, mux baca/tulis per bank
rtl/mem/ntt_core_c1.sv	Salinan byte-untuk-byte ntt_core.sv (Fase 1) dengan poly_mem diganti poly_mem_banked #(NUM_BANKS(1))

Selama pembangkitan RTL, ditemukan dan diperbaiki 5 bug (label case salah lebar bit, ternary lebar port cacat, seleksi runtime-case diganti generate-if elaborasi karena Verilator menge-flag WIDTHEXPAND pada cabang mati, dan — paling signifikan — **Icarus Verilog menolak SystemVerilog function dengan output port** walau Verilator/slang menerimanya, sehingga generator direstrukturisasi memakai always_comb+case murni tanpa function sama sekali).

6. Verifikasi RTL

Gerbang	Hasil
cocotb: bank_map_rom (L=1,2,4,8) + ntt_core_c1	12/12 lulus, di Icarus DAN Verilator
Siklus C1 vs C0 (invariansi konstan-waktu tetap terjaga)	SAMA PERSIS: NTT=897, INTT=1153 — 0 stall cycle dari memori ter-bank
Formal ulang keamanan FSM pada ntt_core_c1	LULUS, k-induction depth 6 — memastikan penggantian modul tidak mengubah perilaku FSM
Regresi Fase 0 (model emas) + Fase 1 (C0)	23/23 dan 10/10 tetap lulus, tidak terganggu oleh Fase 2

7. Hasil Quartus: C1 dibanding C0 (MEASURED)

Compile kernel-only, device dan constraint SDC sama persis dengan C0 (20,000 ns provisional, belum disetujui tim), metodologi identik.

Besaran	C0 (Fase 1)	C1 (Fase 2)	Perubahan
ALM	7.010	6.749	-261 ALM
Register	3.104	3.105	+1
Blok M10K	0 / 553	0 / 553	tidak berubah — lihat §8
Blok DSP	3	3	tidak berubah
Fmax, corner Slow 100°C	14,64 MHz	14,99 MHz	+0,35 MHz
Slack setup terburuk	-48,323 ns	-46,720 ns	membaik 1,6 ns, TETAP GAGAL
Slack hold terburuk	0,211 ns (terpenuhi)	0,168 ns (terpenuhi)	tetap terpenuhi
Siklus NTT / INTT (simulasi)	897 / 1153	897 / 1153	0 — identik

Jalur terburuk C1: layer_q[0] → poly_mem_banked:u_mem|g_bank[0].mem[28][1], delay jalur data 66,087 ns (vs 67,684 ns di C0), tetap didominasi 252 referensi ke sel lpm_divide yang sama — akar masalah persis sama dengan C0, karena Fase 2 memang tidak menyentuh rtl/ntt/modmul_reduce.sv atau aritmetika manapun. Perbaikan kecil slack (~1,6 ns) berasal dari perbedaan penempatan/routing di sekitar modul memori yang ditukar, **bukan** perbaikan timing yang disengaja. Memperbaiki ini adalah lingkup Fase 4 (pipelining) / Fase 5 (aritmetika), sesuai docs/ROADMAP.md.

8. Temuan Jujur: M10K Belum Tercapai

Judul implementasi Fase 2 di roadmap adalah "M10K polynomial storage", tapi hasil terukur tetap **0/553 blok RAM** — identik dengan C0. Log Quartus (Analysis & Synthesis) mengonfirmasi penyebabnya secara eksplisit dan dikutip apa adanya:

```
Info (276014): Found 5 instances of uninferred RAM logic
Info (276007): RAM logic "poly_mem_banked:u_mem|bank_map_rom:u_map_b|Ram0"
is uninferred due to asynchronous read logic
Info (276007): RAM logic "poly_mem_banked:u_mem|bank_map_rom:u_map_a|Ram0"
is uninferred due to asynchronous read logic
Info (276004): RAM logic "twiddle_rom:u_rom|rom_zeta"
is uninferred due to inappropriate RAM size
```

Hanya `bank_map_rom` dan `twiddle_rom` yang disebut di log ini — array penyimpanan 256-entri milik `poly_mem_banked` sendiri bahkan **tidak dikenali** sebagai kandidat RAM. Jumlah register yang nyaris tidak berubah (3104 → 3105) mengonfirmasi penyimpanan itu tetap dibangun dari flip-flop, persis seperti `poly_mem.sv` di C0. Penurunan -261 ALM karenanya dikaitkan dengan logika `bank_map_rom` yang kecil/murah dan pengepakan flip-flop/mux yang berbeda di sekitar modul yang ditukar — **bukan** penghematan blok RAM.

Akar masalah: baik `poly_mem.sv` (C0) maupun `poly_mem_banked.sv` (C1) memakai port baca *asinkron/kombinasi*, supaya satu butterfly (baca kedua operand, hitung, tulis kedua hasil) bisa selesai dalam satu siklus clock tanpa tahap latensi baca. Inferensi M10K Quartus mensyaratkan baca *sinkron* — persis alasan yang dikutip pesan 276007 di atas. Mencapai M10K butuh desain yang membaca operand ke register satu siklus sebelum dipakai — perubahan penjadwalan FSM, **bukan** sesuatu yang ditambah diam-diam ke C1 tanpa tinjauan tim, karena ini akan mengubah jumlah siklus yang sekarang terbukti identik dengan C0 (897/1153).

Ini dicatat sebagai keterbatasan nyata, bukan ditambah diam-diam. Kriteria PASS Fase 2 sendiri di `docs/ROADMAP.md` (bebas-tabrakan terbukti untuk semua L; stall cycles=0 pada L=1; bit-exact; siklus konstan; baris C1 terisi) **tidak mensyaratkan** penggunaan M10K bukan-nol — angka 0 yang jujur itulah yang diisikan ke baris C1, tanpa melemahkan kata-kata kriteria.

9. Status

Fase 2 **PARTIAL**. Kelima kriteria PASS milik Fase 2 sendiri terpenuhi (bebas-tabrakan, stall=0, bit-exact, siklus konstan, baris C1 terisi), tapi CRG-9 tetap FAIL (timing belum terpenuhi, sama akar masalah dengan Fase 1) dan M10K belum tercapai. Kotak Approval pada `docs/results/result_phase2.md` belum dicentang, menunggu tinjauan tim.

10. Keputusan Terbuka untuk Tim

- ADR target clock resmi untuk C0/C1 — 20 ns yang dipakai sekarang murni provisional.
- Kejar M10K sekarang (butuh ubah jadwal FSM, tambah 1 tahap register baca) atau tunda ke fase lain.
- Cara menutup Fase 1 & 2 yang timing-nya gagal: diterima sebagai baseline terdokumentasi (perbaikan menyusul di Fase 4/5) atau diperbaiki dulu sebelum dibekukan.
- Proses gerbang fase: apakah boleh mulai fase berikutnya sebelum kotak Approval fase sebelumnya dicentang (terjadi antara Fase 1→2) — perlu ditertibkan atau diterima sebagai preseden.

11. Cara Reproduksi

```
. scripts/env.sh
python3 scripts/gen_bank_map.py
python3 tb/mem/run_mem_tests.py icarus
python3 tb/mem/run_mem_tests.py verilator
sby -f formal/phase02-mem/bank_map_safety.sby
sby -f formal/phase02-mem/ntt_core_c1_safety.sby
# Quartus C1: quartus/phase02_mem_c1/phase02_mem_c1.qpf (dijalankan manual oleh tim mulai Fase 3)
```